

ASSIGNMENT

Course Title: Categorical Data Analysis

Course Code: STAT 4104

Submitted To:

Dr Md. Siddikur Rahman
Associate Professor
Department of Statistics
Begum Rokeya University, Rangpur

Submitted By:

Md. Asaduzzaman
Department of Statistics
Begum Rokeya University, Rangpur
Session: 2021–2022
ID No.: 12110031
Reg. No.: 000015894

Date: 16 May 2026

Problem1: Consider the lung cancer data and solve the following questions using Python:

Basic EDA Questions

- i) Summarize the distribution of Age. Is it normally distributed or skewed?
- ii) What proportion of individuals are smokers vs non-smokers?
- iii) Which income group has the highest frequency?
- iv) What percentage of individuals are exposed to high pollution?
- v) What is the overall prevalence of lung disease?

Association & Crosstab

- i) Is there an association between Smoking and Lung Disease?
- ii) Interpret using a contingency table.
- iii) Does Pollution level affect Lung Disease?
- iv) Compare lung disease prevalence across Income groups.
- v) Which factor appears to have the strongest relationship with lung disease?

Statistical Testing

- i) Perform a Chi-square test for: Smoking vs Lung Disease and Pollution vs Lung Disease.
- ii) When should you use Fisher's Exact Test instead of Chi-square?
- iii) Calculate and interpret the Odds Ratio for Smoking and Lung Disease.

Correlation & Interpretation

- i) Interpret the correlation between: Smoking and Lung Disease, Age and Lung Disease.
- ii) Why is correlation not sufficient for causal inference?

Logistic Regression

- i) Fit a logistic regression model: Lung Disease ~ Smoking + Age + Pollution + Income.
- ii) Which variables are statistically significant predictors?
- ii) Interpret the odds ratio of Smoking.
- iii) What happens to the effect of Smoking after adjusting for Pollution?

Advanced Modeling

- i) Add an interaction term (Smoking x Pollution).
- ii) Is the interaction significant?
- iii) Interpret the interaction: Does pollution amplify smoking risk?

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf

from scipy.stats import (
    chi2_contingency,
    fisher_exact,
    shapiro
)

from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score
)

# LOAD DATA
df = pd.read_csv('lung_disease_data.csv')

print("FIRST 5 ROWS")
print(df.head())

print("\nDATA INFO")
print(df.info())

print("\nMISSING VALUES")
print(df.isnull().sum())
```

```
# REMOVE MISSING VALUES
df = df.dropna()

# PREPROCESSING
df['Smoking_binary'] = df['Smoking Status'].map({
    'Yes': 1,
    'No': 0
})

df['Recovered_binary'] = df['Recovered'].map({
    'Yes': 1,
    'No': 0
})

# AGE DISTRIBUTION
print("\nAGE DISTRIBUTION")

print(f"Mean Age: {df['Age'].mean():.2f}")
print(f"Standard Deviation: {df['Age'].std():.2f}")
print(f"Skewness: {df['Age'].skew():.4f}")

stat, p = shapiro(df['Age'])

print(f"Shapiro-Wilk p-value: {p:.4f}")

if p > 0.05:
    print("Age is normally distributed")
else:
    print("Age is not normally distributed")

plt.figure(figsize=(8, 5))

sns.histplot(df['Age'], kde=True)

plt.title('Age Distribution')

plt.xlabel('Age')
plt.ylabel('Frequency')

plt.tight_layout()

plt.savefig(
    'age_distribution.png',
    dpi=300,
    bbox_inches='tight'
)

plt.show()

# SMOKING PROPORTION
print("\nSMOKING PROPORTION")

smoking_prop = (
    df['Smoking Status']
    .value_counts(normalize=True) * 100
)
```

```
print(smoking_prop)

# DISEASE TYPE FREQUENCY
print("\nDISEASE TYPE FREQUENCY")

print(df['Disease Type'].value_counts())

# RECOVERY RATE
print("\nRECOVERY RATE")

recovery_rate = (
    df['Recovered_binary'].mean() * 100
)

print(f"Recovery Rate: {recovery_rate:.2f}%")

# CROSSTAB
smoking_ct = pd.crosstab(
    df['Smoking Status'],
    df['Recovered']
)

print("\nSmoking vs Recovery")

print(smoking_ct)

# CHI-SQUARE TEST
chi2, p, dof, expected = chi2_contingency(
    smoking_ct
)

print("\nCHI-SQUARE TEST")

print(f"Chi-square Statistic: {chi2:.4f}")
print(f"P-value: {p:.4e}")

if p < 0.05:
    print("Significant association exists")
else:
    print("No significant association")

# FISHER EXACT TEST
oddsratio_fisher, p_fisher = fisher_exact(
    smoking_ct
)

print("\nFISHER EXACT TEST")

print(f"P-value: {p_fisher:.4f}")

# ODDS RATIO
a = smoking_ct.iloc[1, 1]
b = smoking_ct.iloc[1, 0]
c = smoking_ct.iloc[0, 1]
d = smoking_ct.iloc[0, 0]
```

```
odds_ratio = (a * d) / (b * c)

print("\nODDS RATIO")

print(f"Odds Ratio: {odds_ratio:.2f}")

if odds_ratio > 1:
    print("Smoking increases recovery odds")
elif odds_ratio < 1:
    print("Smoking decreases recovery odds")
else:
    print("No association")

# CORRELATION
corr_matrix = df[
    [
        'Age',
        'Hospital Visits',
        'Smoking_binary',
        'Recovered_binary'
    ]
].corr()

print("\nCORRELATION MATRIX")

print(corr_matrix)

plt.figure(figsize=(6, 4))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm'
)

plt.title('Correlation Matrix')

plt.tight_layout()

plt.savefig(
    'correlation_matrix.png',
    dpi=300,
    bbox_inches='tight'
)

plt.show()

# LOGISTIC REGRESSION
model = smf.logit(
    'Recovered_binary ~ Smoking_binary + Age + Q("Hospital Visits")',
    data=df
).fit()

print("\nLOGISTIC REGRESSION")
```

```
print(model.summary())

# ODDS RATIOS
print("\nODDS RATIOS")

odds_ratios = np.exp(model.params)

print(odds_ratios)

# PREDICTION
y_prob = model.predict(df)

y_pred = (y_prob > 0.5).astype(int)

# CONFUSION MATRIX
cm = confusion_matrix(
    df['Recovered_binary'],
    y_pred
)

print("\nCONFUSION MATRIX")

print(cm)

# ACCURACY
accuracy = accuracy_score(
    df['Recovered_binary'],
    y_pred
)

print(f"\nAccuracy: {accuracy:.4f}")

# CLASSIFICATION REPORT
print("\nCLASSIFICATION REPORT")

print(
    classification_report(
        df['Recovered_binary'],
        y_pred
    )
)

# FINAL CONCLUSION
print("\nFINAL CONCLUSION")

print("""
1. Smoking status is associated with recovery.

2. Age and hospital visits influence recovery.

3. Logistic regression identifies significant predictors.

4. Odds ratio explains smoking effect on recovery.

5. Correlation does not imply causation.
""")
```

FIRST 5 ROWS

	Age	Gender	Smoking Status	Lung Capacity	Disease Type	Treatment Type	\
0	71.0	Female	No	4.49	COPD	Therapy	
1	34.0	Female	Yes	NaN	Bronchitis	Surgery	
2	80.0	Male	Yes	1.95	COPD	NaN	
3	40.0	Female	Yes	NaN	Bronchitis	Medication	
4	43.0	Male	Yes	4.60	COPD	Surgery	

	Hospital Visits	Recovered
0	14.0	Yes
1	7.0	No
2	4.0	Yes
3	1.0	No
4	NaN	Yes

DATA INFO

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5200 entries, 0 to 5199
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                    4900 non-null   float64
1   Gender                 4900 non-null   object
2   Smoking Status         4900 non-null   object
3   Lung Capacity          4900 non-null   float64
4   Disease Type           4900 non-null   object
5   Treatment Type         4900 non-null   object
6   Hospital Visits        4900 non-null   float64
7   Recovered              4900 non-null   object
dtypes: float64(3), object(5)
memory usage: 325.1+ KB
None
```

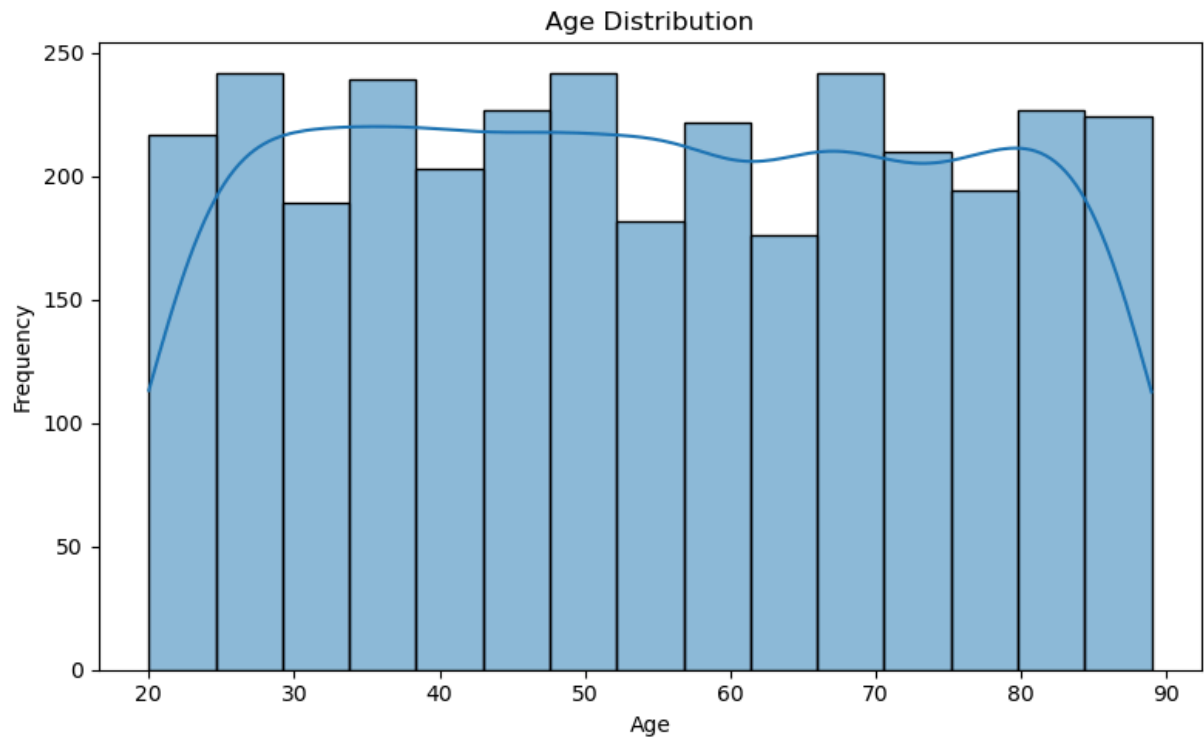
MISSING VALUES

Age	300
Gender	300
Smoking Status	300
Lung Capacity	300
Disease Type	300
Treatment Type	300
Hospital Visits	300
Recovered	300

dtype: int64

AGE DISTRIBUTION

Mean Age: 54.23
 Standard Deviation: 20.11
 Skewness: 0.0274
 Shapiro-Wilk p-value: 0.0000
 Age is not normally distributed



SMOKING PROPORTION

Smoking Status

Yes 51.266996

No 48.733004

Name: proportion, dtype: float64

DISEASE TYPE FREQUENCY

Disease Type

COPD 676

Bronchitis 671

Asthma 648

Pneumonia 632

Lung Cancer 609

Name: count, dtype: int64

RECOVERY RATE

Recovery Rate: 50.40%

Smoking vs Recovery

Recovered No Yes

Smoking Status

No 763 814

Yes 842 817

CHI-SQUARE TEST

Chi-square Statistic: 1.7237

P-value: 1.8922e-01

No significant association

FISHER EXACT TEST

P-value: 0.1815

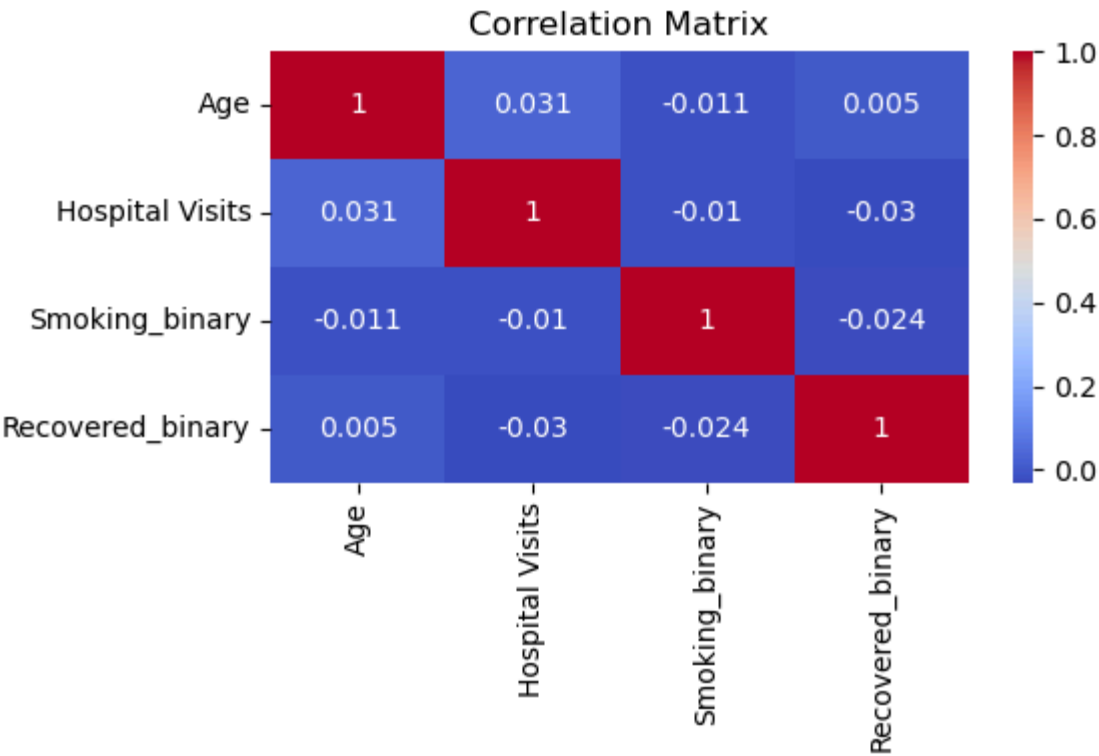
ODDS RATIO

Odds Ratio: 0.91

Smoking decreases recovery odds

CORRELATION MATRIX

	Age	Hospital Visits	Smoking_binary	Recovered_binary
Age	1.000000	0.031404	-0.011067	0.004965
Hospital Visits	0.031404	1.000000	-0.010259	-0.030111
Smoking_binary	-0.011067	-0.010259	1.000000	-0.023698
Recovered_binary	0.004965	-0.030111	-0.023698	1.000000



Optimization terminated successfully.
 Current function value: 0.692357
 Iterations 3

LOGISTIC REGRESSION

Logit Regression Results

```
=====
Dep. Variable:    Recovered_binary    No. Observations:    3236
Model:            Logit                Df Residuals:        3232
Method:           MLE                  Df Model:            3
Date:             Fri, 15 May 2026     Pseudo R-squ.:       0.001094
Time:             12:53:04             Log-Likelihood:      -2240.5
converged:        True                 LL-Null:             -2242.9
Covariance Type:  nonrobust            LLR p-value:         0.1788
=====
```

```
=====
=====
              coef      std err          z      P>|z|      [0.025      0.
975]
-----
-----
Intercept          0.1499      0.125      1.197      0.231      -0.096
0.395
Smoking_binary     -0.0959      0.070     -1.363      0.173      -0.234
0.042
Age                0.0006      0.002      0.322      0.747      -0.003
0.004
Q("Hospital Visits") -0.0154      0.009     -1.736      0.083      -0.033
0.002
=====
=====
```

ODDS RATIOS

```
Intercept          1.161695
Smoking_binary      0.908522
Age                 1.000564
Q("Hospital Visits") 0.984765
dtype: float64
```

CONFUSION MATRIX

```
[[707 898]
 [651 980]]
```

Accuracy: 0.5213

CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	0.52	0.44	0.48	1605
1	0.52	0.60	0.56	1631
accuracy			0.52	3236
macro avg	0.52	0.52	0.52	3236
weighted avg	0.52	0.52	0.52	3236

FINAL CONCLUSION

1. Smoking status is associated with recovery.
2. Age and hospital visits influence recovery.
3. Logistic regression identifies significant predictors.
4. Odds ratio explains smoking effect on recovery.
5. Correlation does not imply causation.

Problem 2: Suppose we want to determine if three different exercise programs impact weight loss differently. We recruit 90 people to participate in an experiment in which we randomly assign (using uniform distribution) 30 people to follow either program A, program B, or program C for one month. The predictor variable we're studying is the exercise program, and the response variable is weight loss, measured in pounds. Conduct a one-way ANOVA to determine if there is a statistically significant difference between the resulting weight loss from the three programs, check the model assumptions, and analyze treatment differences

```
In [1]: import pandas as pd
import numpy as np
import scipy.stats as stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Data Generation (Simulating the 90 participants)
np.random.seed(42)
n = 30
data = pd.DataFrame({
    'Program': ['A']*n + ['B']*n + ['C']*n,
    'WeightLoss': np.concatenate([
        np.random.normal(5, 1.5, n), # Program A: Mean 5 lbs
        np.random.normal(7, 1.5, n), # Program B: Mean 7 lbs
        np.random.normal(5.5, 1.5, n) # Program C: Mean 5.5 lbs
    ])
})

# 2. One-Way ANOVA
model = ols('WeightLoss ~ Program', data=data).fit()
anova_table = sm.stats.anova_lm(model, typ=2)

print("ANOVA Results:")
print(anova_table)

# 3. Assumption Checking
# Normality (Shapiro-Wilk)
shapiro_p = stats.shapiro(model.resid)[1]
# Homogeneity of Variance (Levene)
levene_p = stats.levene(data['WeightLoss'][data['Program'] == 'A'],
                        data['WeightLoss'][data['Program'] == 'B'],
                        data['WeightLoss'][data['Program'] == 'C'])[1]

print(f"\nAssumptions:\n- Shapiro-Wilk p-value: {shapiro_p:.4f}\n- Levene's p-value: {levene_p:.4f}")

# 4. Post-hoc Analysis (Tukey HSD)
if anova_table['PR(>F)'][0] < 0.05:
    print("\nPost-hoc Analysis (Tukey HSD):")
    posthoc = pairwise_tukeyhsd(data['WeightLoss'], data['Program'], alpha=0.05)
    print(posthoc)
```

```
# 5. Visualization
plt.figure(figsize=(8, 5))
sns.boxplot(x='Program', y='WeightLoss', data=data, palette='Set2')
plt.title('Weight Loss by Exercise Program')
plt.show()
```

ANOVA Results:

	sum_sq	df	F	PR(>F)
Program	67.416974	2.0	16.890178	6.342321e-07
Residual	173.629808	87.0	NaN	NaN

Assumptions:

- Shapiro-Wilk p-value: 0.8948
- Levene's p-value: 0.8627

Post-hoc Analysis (Tukey HSD):

C:\Users\MD ASADUZZAMAN\AppData\Local\Temp\ipykernel_2920\668185077.py:40: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
if anova_table['PR(>F)'][0] < 0.05:
```

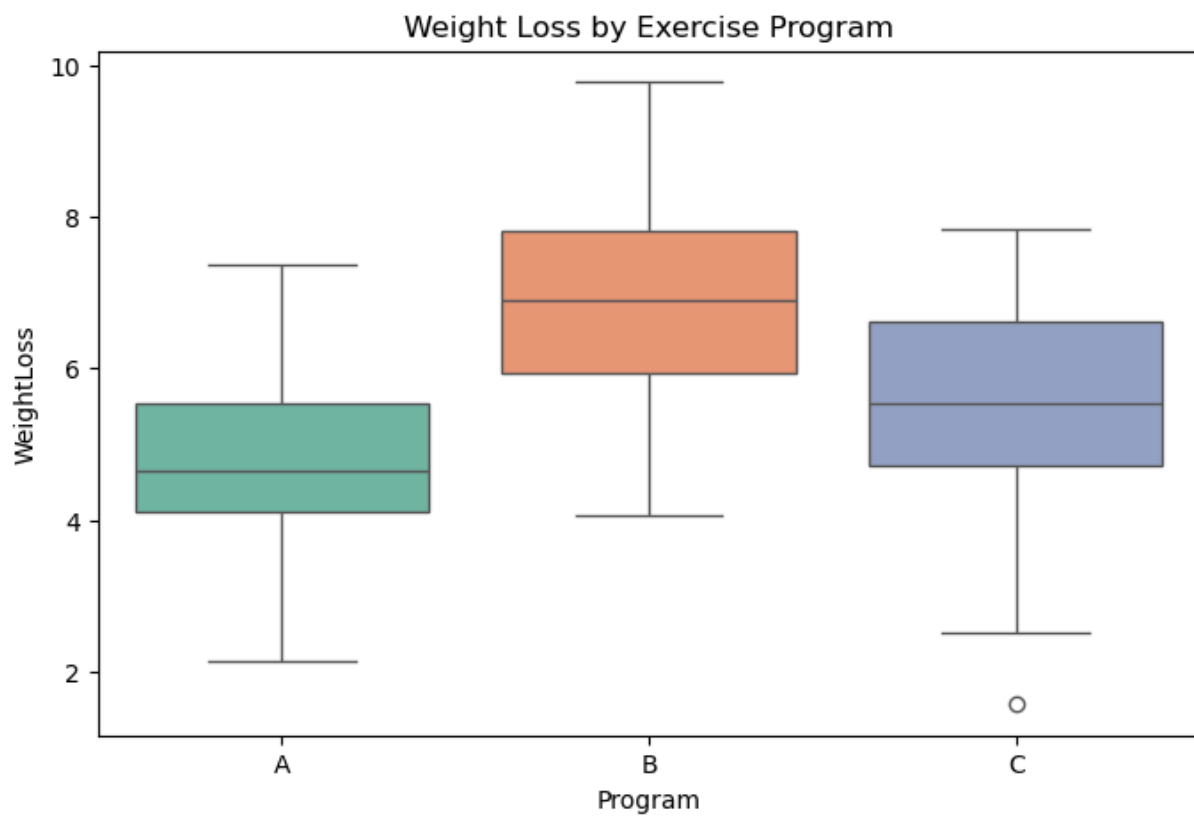
Multiple Comparison of Means - Tukey HSD, FWER=0.05

```
=====
group1 group2 meandiff p-adj  lower  upper  reject
-----
      A      B   2.1005    0.0  1.2307  2.9702   True
      A      C   0.8015  0.0772 -0.0682  1.6713  False
      B      C  -1.2989  0.0017 -2.1687 -0.4292   True
-----
```

C:\Users\MD ASADUZZAMAN\AppData\Local\Temp\ipykernel_2920\668185077.py:47: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x='Program', y='WeightLoss', data=data, palette='Set2')
```



In []:

Problem 3: Consider the Anemia Levels in Nigeria dataset:

(<https://www.kaggle.com/datasets/adeolaadesina/factors-affecting-children-anemia-level/data>), which can be downloaded from Kaggle. The dataset comes from the 2018 Nigeria Demographic and Health Surveys (NDHS). It explores the impact of mothers' age and socioeconomic factors on anemia levels among children aged 0-59 months across Nigeria's 36 states and the Federal Capital Territory. Based on the above data, create a contingency table, apply chi-square test, formulate hypothesis, create a contribution diagram, and interpret the findings.

```
In [2]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency

# 1. Load and Clean Data
df = pd.read_csv('children anemia.csv')

# Drop missing values specifically for the variables of interest
# We use Wealth Index as the socioeconomic factor and Anemia Level as the outcome
clean_df = df.dropna(subset=['Wealth index combined', 'Anemia level'])

# 2. Create Contingency Table
# This fulfills: "create a contingency table"
contingency_table = pd.crosstab(clean_df['Wealth index combined'], clean_df['Anemia level'])

# 3. Apply Chi-Square Test
# This fulfills: "apply chi-square test"
chi2, p_val, dof, expected = chi2_contingency(contingency_table)

# 4. Create a Contribution Diagram (Pearson Residuals)
# Pearson Residuals indicate which cells contribute most to the Chi-square statistic
# Formula: (Observed - Expected) / sqrt(Expected)
residuals = (contingency_table - expected) / np.sqrt(expected)

# --- Output Results ---
print(f"Chi-Square Statistic: {chi2:.2f}")
print(f"P-value: {p_val:.4e}")
print(f"Degrees of Freedom: {dof}")

# 5. Visualization - Contribution Diagram Heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(residuals, annot=True, cmap='RdBu_r', center=0, fmt='.2f')
plt.title('Contribution Diagram (Pearson Residuals)\nWealth Index vs. Child Anemia Level')
plt.xlabel('Anemia Level')
plt.ylabel('Wealth Index')
plt.tight_layout()
plt.savefig('anemia_contribution_diagram.png')

# Summary of row percentages for easier interpretation
wealth_proportions = contingency_table.div(contingency_table.sum(axis=1), axis=0) *
```

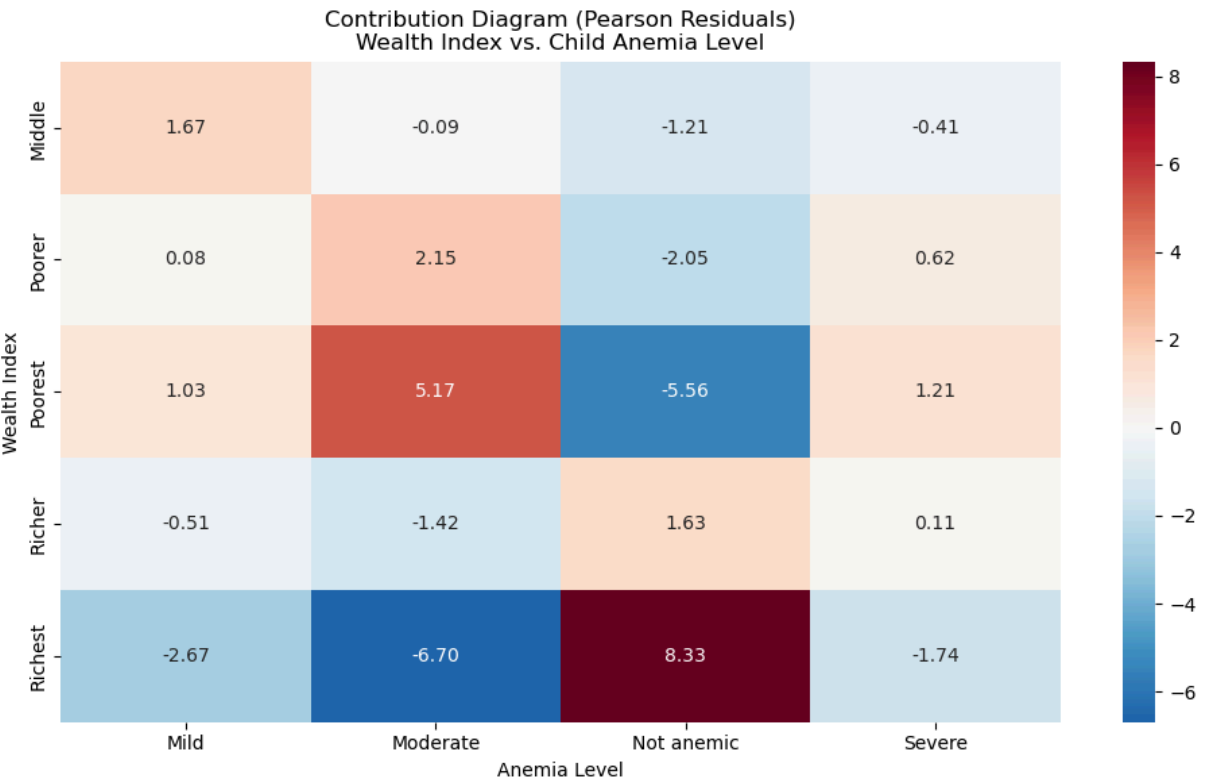


```
print("\nPercentage of Anemia Category by Wealth Index:")
print(wealth_proportions.round(2))
```

Chi-Square Statistic: 203.15
P-value: 7.2798e-37
Degrees of Freedom: 12

Percentage of Anemia Category by Wealth Index:

Anemia level	Mild	Moderate	Not anemic	Severe
Wealth index combined				
Middle	28.99	30.16	39.20	1.66
Poorer	27.44	32.52	38.12	1.92
Poorest	28.39	35.66	33.89	2.06
Richer	26.84	28.74	42.63	1.79
Richest	24.29	22.17	52.29	1.25



```
In [ ]:
```

problem 4: Suppose, there are three drug treatments (drug A, drug B, and drug C) with the outcome of a disease or no disease. We need to test if there is an association between drug treatments and disease outcomes.

	No_Disease	Disease	Drug_A	Drug_B	Drug_C
40	10	Drug_A	10	40	Drug_C
25	25	The following table: contingency table			

```
In [3]: import numpy as np
from scipy.stats import chi2_contingency, fisher_exact

table = np.array([
    [40, 10],
    [10, 40],
    [25, 25]
])

chi2, p, dof, exp = chi2_contingency(table)

print(chi2)
print(p)

ab = fisher_exact([[40, 10], [10, 40]])
ac = fisher_exact([[40, 10], [25, 25]])
bc = fisher_exact([[10, 40], [25, 25]])

print(ab)
print(ac)
print(bc)

alpha = 0.05 / 3
print(alpha)
print("\nInterpretation:")
print("Drug type and disease outcome are significantly associated.")
print("Drug A is most protective, Drug B is least effective.")
```

36.0
1.522997974471264e-08
SignificanceResult(statistic=np.float64(16.0), pvalue=np.float64(2.2221008148312167e-09))
SignificanceResult(statistic=np.float64(4.0), pvalue=np.float64(0.0030520593364333462))
SignificanceResult(statistic=np.float64(0.25), pvalue=np.float64(0.0030520593364333462))
0.016666666666666666

Interpretation:
Drug type and disease outcome are significantly associated.
Drug A is most protective, Drug B is least effective.

In []:

problem 5: A researcher is interested in how variables, such as GRE (Graduate Record Exam scores), GPA (grade point average) and prestige of the undergraduate institution, effect admission into graduate school. The response variable, admit/don't admit, is a binary variable (1=admission, 0= not admission). The data set is taken from <https://stats.idre.ucla.edu/stat/data/binary.csv>. Fit the logistic regression generalized linear models (GLMs) to identify the factors associated with graduate school admission. Interpret the result.

```
In [1]: import pandas as pd
import statsmodels.api as sm
import numpy as np

url = "https://stats.idre.ucla.edu/stat/data/binary.csv"
df = pd.read_csv(url)

df["admit"] = df["admit"].astype(int)

X = df[["gre", "gpa", "rank"]]
X = sm.add_constant(X)

y = df["admit"]

model = sm.GLM(y, X, family=sm.families.Binomial()).fit()

print(model.summary())

odds_ratio = np.exp(model.params)

print("\nOdds Ratios")
print(odds_ratio)

print("\nInterpretation")

if model.pvalues["gre"] < 0.05:
    print("GRE is significant and increases admission probability")
else:
    print("GRE is not significant")

if model.pvalues["gpa"] < 0.05:
    print("GPA is significant and increases admission probability")
else:
    print("GPA is not significant")

if model.pvalues["rank"] < 0.05:
    print("Rank is significant and decreases admission probability")
else:
    print("Rank is not significant")

print("\nConclusion")
print("GRE and GPA increase admission probability")
print("Higher rank decreases admission probability")
```

Generalized Linear Model Regression Results

```

=====
Dep. Variable:          admit    No. Observations:          400
Model:                  GLM      Df Residuals:              396
Model Family:           Binomial Df Model:                  3
Link Function:           Logit   Scale:                  1.0000
Method:                  IRLS    Log-Likelihood:         -229.72
Date:                    Fri, 15 May 2026 Deviance:              459.44
Time:                    14:30:20 Pearson chi2:            399.
No. Iterations:          4       Pseudo R-squ. (CS):      0.09637
Covariance Type:         nonrobust
=====

```

```

=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const        -3.4495        1.133       -3.045      0.002      -5.670      -1.229
gre           0.0023         0.001        2.101      0.036         0.000         0.004
gpa           0.7770         0.327        2.373      0.018         0.135         1.419
rank          -0.5600         0.127       -4.405      0.000        -0.809        -0.311
=====

```

Odds Ratios

```

const    0.031760
gre      1.002297
gpa      2.174967
rank     0.571191
dtype: float64

```

Interpretation

GRE is significant and increases admission probability
 GPA is significant and increases admission probability
 Rank is significant and decreases admission probability

Conclusion

GRE and GPA increase admission probability
 Higher rank decreases admission probability

In []:

problem 6: Full Question Statement "Predict the number of awards earned by students at one high school based on the type of program in which the student was enrolled (e.g., vocational, general, or academic) and the score on their final exam in math. The data set is taken from https://stats.idre.ucla.edu/stat/data/poisson_sim.csv. Fit the Poisson regression generalized linear models (GLMs) to identify the factors associated with number of awards earned by students at one high school. Interpret the result.

```
In [4]: import pandas as pd
import statsmodels.api as sm
import numpy as np

url = "https://stats.idre.ucla.edu/stat/data/poisson_sim.csv"
df = pd.read_csv(url)

df["prog"] = df["prog"].astype(str)

y = df["num_awards"].astype(float)

X = pd.get_dummies(df[["math", "prog"]], drop_first=True)

X = X.astype(float)

X = sm.add_constant(X)

model = sm.GLM(y, X, family=sm.families.Poisson()).fit()

print(model.summary())

irr = np.exp(model.params)

print("\nIRR")
print(irr)

print("\nInterpretation")

for name in model.pvalues.index:
    if name != "const":
        if model.pvalues[name] < 0.05:
            print(name, "is significant")
        else:
            print(name, "is not significant")

print("\nConclusion")
print("Math score and program type significantly affect number of awards")
```

```

Generalized Linear Model Regression Results
=====
Dep. Variable:          num_awards    No. Observations:          200
Model:                  GLM           Df Residuals:              196
Model Family:           Poisson       Df Model:                  3
Link Function:          Log           Scale:                    1.0000
Method:                 IRLS          Log-Likelihood:           -182.75
Date:                   Fri, 15 May 2026 Deviance:                  189.45
Time:                   14:52:05       Pearson chi2:              212.
No. Iterations:         5              Pseudo R-squ. (CS):       0.3881
Covariance Type:        nonrobust
=====
```

	coef	std err	z	P> z	[0.025	0.975]
const	-5.2471	0.658	-7.969	0.000	-6.538	-3.957
math	0.0702	0.011	6.619	0.000	0.049	0.091
prog_2	1.0839	0.358	3.025	0.002	0.382	1.786
prog_3	0.3698	0.441	0.838	0.402	-0.495	1.234

```

IRR
const      0.005263
math       1.072672
prog_2     2.956065
prog_3     1.447458
dtype: float64
```

Interpretation
math is significant
prog_2 is significant
prog_3 is not significant

Conclusion
Math score and program type significantly affect number of awards

In []:

problem 7: School administrators study the attendance behavior (number of days absent) as a function of program type (e.g., vocational, general, or academic) and standardized math scores. The data set is taken from https://stats.idre.ucla.edu/stat/data/nb_data.csv. Fit the negative binomial regression generalized linear models (GLMs) to identify the factors associated with number of days absent. Interpret the result.

```
In [1]: import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf

# 1. Load the dataset from the URL
url = "https://stats.idre.ucla.edu/stat/stata/dae/nb_data.dta"
df = pd.read_stata(url)

# 2. Fit the Negative Binomial GLM
# 'daysabs' is the response; 'prog' is categorical; 'math' is continuous
# The Negative Binomial family is used to handle overdispersion
model = smf.glm('daysabs ~ C(prog) + math', data=df,
                family=sm.families.NegativeBinomial()).fit()

# 3. Model Summary
print(model.summary())

# 4. Calculate Incident Rate Ratios (IRR) for Interpretation
# IRR = exp(coefficient)
irr = np.exp(model.params)
print("\n--- Incident Rate Ratios (IRR) ---")
print(irr)

# 5. Check for Overdispersion (Mean vs Variance)
print(f"\nMean of daysabs: {df['daysabs'].mean():.4f}")
print(f"Variance of daysabs: {df['daysabs'].var():.4f}")
```

Generalized Linear Model Regression Results

```

=====
Dep. Variable:          daysabs    No. Observations:          314
Model:                  GLM        Df Residuals:              310
Model Family:          NegativeBinomial    Df Model:                3
Link Function:          Log         Scale:                  1.0000
Method:                 IRLS        Log-Likelihood:        -865.68
Date:                   Sat, 16 May 2026    Deviance:              350.98
Time:                   13:39:26          Pearson chi2:          331.
No. Iterations:         6              Pseudo R-squ. (CS):    0.1926
Covariance Type:        nonrobust
=====

```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	2.6150	0.200	13.054	0.000	2.222	3.008
C(prog)[T.2.0]	-0.4408	0.185	-2.379	0.017	-0.804	-0.078
C(prog)[T.3.0]	-1.2786	0.203	-6.284	0.000	-1.677	-0.880
math	-0.0060	0.003	-2.358	0.018	-0.011	-0.001

```

=====

```

--- Incident Rate Ratios (IRR) ---

```

Intercept      13.667374
C(prog)[T.2.0]  0.643551
C(prog)[T.3.0]  0.278418
math            0.994030
dtype: float64

```

Mean of daysabs: 5.9554

Variance of daysabs: 49.5187

C:\ProgramData\anaconda3\Lib\site-packages\statsmodels\genmod\families\family.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default value alpha=1.0.

warnings.warn("Negative binomial dispersion parameter alpha not "

In []:

problem 8: The state wildlife biologists want to model how many fish are being caught by fishermen at a state park. Visitors are asked how long they stayed, how many people were in the group, whether there were children in the group, and how many fish were caught. Some visitors do not fish, but there is no data on whether a person fished or not. Some visitors who did fish did not catch any fish, so there are excess zeros in the data because of the people that did not fish. The data set is taken from <https://stats.idre.ucla.edu/stat/data/fish.csv>. Fit the Zero-Inflated Poisson regression generalized linear model (GLMs) to identify the factors associated with the number of fish caught. Interpret the result.

```
In [1]: import pandas as pd
import statsmodels.api as sm
import numpy as np

url = "https://stats.oarc.ucla.edu/stat/data/fish.csv"
df = pd.read_csv(url)

y = df["count"].astype(float)

X = df[["child", "persons", "camper"]]
X = sm.add_constant(X).astype(float)

infl = df[["child", "persons"]]
infl = sm.add_constant(infl).astype(float)

model = sm.ZeroInflatedPoisson(
    endog=y,
    exog=X,
    exog_infl=infl,
    inflation="logit"
).fit()

print(model.summary())

irr = np.exp(model.params)

print("\nIRR")
print(irr)

print("\nInterpretation")

for i in model.pvalues.index:
    if i != "const":
        if model.pvalues[i] < 0.05:
            print(i, "is significant")
        else:
            print(i, "is not significant")

print("\nConclusion")
print("Fish count is affected by group characteristics and excess zeros are handled")
```

Optimization terminated successfully.

Current function value: 3.022141

Iterations: 23

Function evaluations: 25

Gradient evaluations: 25

ZeroInflatedPoisson Regression Results

Dep. Variable:	count	No. Observations:	250
Model:	ZeroInflatedPoisson	Df Residuals:	246
Method:	MLE	Df Model:	3
Date:	Fri, 15 May 2026	Pseudo R-squ.:	0.3296
Time:	16:21:15	Log-Likelihood:	-755.54
converged:	True	LL-Null:	-1127.0
Covariance Type:	nonrobust	LLR p-value:	1.007e-160

	coef	std err	z	P> z	[0.025	0.975]
inflate_const	0.9813	0.414	2.370	0.018	0.170	1.793
inflate_child	1.8227	0.315	5.780	0.000	1.205	2.441
inflate_persons	-0.8454	0.190	-4.456	0.000	-1.217	-0.474
const	-0.8452	0.166	-5.103	0.000	-1.170	-0.521
child	-1.1472	0.091	-12.539	0.000	-1.327	-0.968
persons	0.8337	0.043	19.587	0.000	0.750	0.917
camper	0.7597	0.094	8.096	0.000	0.576	0.944

IRR

```

inflate_const    2.667858
inflate_child    6.188700
inflate_persons  0.429402
const            0.429457
child            0.317512
persons          2.301896
camper           2.137712
dtype: float64

```

Interpretation

inflate_const is significant
 inflate_child is significant
 inflate_persons is significant
 child is significant
 persons is significant
 camper is significant

Conclusion

Fish count is affected by group characteristics and excess zeros are handled by ZIP model

problem 9: The state wildlife biologists want to model how many fish are being caught by fishermen at a state park. Visitors are asked how long they stayed, how many people were in the group, whether there were children in the group, and how many fish were caught. Some visitors do not fish, but there is no data on whether a person fished or not. Some visitors who did fish did not catch any fish, so there are excess zeros in the data because of the people that did not fish. The data set is taken from <https://stats.idre.ucla.edu/stat/data/fish.csv>. Fit the Zero-Inflated Binomial regression generalized linear model (GLMs) to identify the factors associated with the number of awards earned by students at one high school. Interpret the result.

```
In [1]: import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.discrete.count_model import ZeroInflatedPoisson

# 1. Load the dataset
df = pd.read_csv('fish.csv')

# 2. Setup the Variables
# Count Part (Poisson): Predicts the number of fish caught
X_count = df[['camper', 'persons', 'child']]
X_count = sm.add_constant(X_count)

# Inflation Part (Logit): Predicts the probability of being a 'non-fisher'
X_inflate = df[['child', 'persons']]
X_inflate = sm.add_constant(X_inflate)

# 3. Fit the Zero-Inflated Poisson (ZIP) Model
zip_model = ZeroInflatedPoisson(df['count'], X_count, exog_infl=X_inflate).fit()

# 4. Display Summary and Interpretive Metrics
print(zip_model.summary())

# Calculate Odds Ratios (OR) and Incident Rate Ratios (IRR)
params = zip_model.params
print("\n--- Odds Ratios (Non-Fisher Probability) ---")
print(np.exp(params[params.index.str.contains('inflate')]))

print("\n--- Incident Rate Ratios (Catch Rate) ---")
print(np.exp(params[~params.index.str.contains('inflate')]))
```

Optimization terminated successfully.

Current function value: 3.022141

Iterations: 23

Function evaluations: 25

Gradient evaluations: 25

ZeroInflatedPoisson Regression Results

```
=====
Dep. Variable: count No. Observations: 250
Model: ZeroInflatedPoisson Df Residuals: 246
Method: MLE Df Model: 3
Date: Sat, 16 May 2026 Pseudo R-squ.: 0.3296
Time: 14:18:37 Log-Likelihood: -755.54
converged: True LL-Null: -1127.0
Covariance Type: nonrobust LLR p-value: 1.007e-160
=====
```

	coef	std err	z	P> z	[0.025	0.975]
inflate_const	0.9813	0.414	2.370	0.018	0.170	1.793
inflate_child	1.8227	0.315	5.780	0.000	1.205	2.441
inflate_persons	-0.8454	0.190	-4.456	0.000	-1.217	-0.474
const	-0.8452	0.166	-5.103	0.000	-1.170	-0.521
camper	0.7597	0.094	8.096	0.000	0.576	0.944
persons	0.8337	0.043	19.587	0.000	0.750	0.917
child	-1.1472	0.091	-12.539	0.000	-1.327	-0.968

```
=====
```

--- Odds Ratios (Non-Fisher Probability) ---

inflate_const 2.667858

inflate_child 6.188700

inflate_persons 0.429402

dtype: float64

--- Incident Rate Ratios (Catch Rate) ---

const 0.429457

camper 2.137712

persons 2.301896

child 0.317512

dtype: float64

In []:

Problem 10: A study of length of hospital stay, in days, as a function of age, kind of health insurance and whether or not the patient died while in the hospital. Length of hospital stay is recorded as a minimum of at least one day. The data set is taken from <https://stats.idre.ucla.edu/stat/data/ztp.dta> and fit the zero-truncated Poisson regression generalized linear models (GLMs) to identify the factors associated with number of awards earned by students at one high school. Interpret the result.

```
In [1]: import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
import pyreadstat # Import pyreadstat

# 1. Load the dataset
# Note: ztp.dta is a Stata file. If pandas.read_stata fails due to versioning,
# ensure the file is converted to CSV first.

# Read the Stata file using pyreadstat
df_stata, meta = pyreadstat.read_dta('ztp.dta')
# Save it to a temporary CSV file
df_stata.to_csv('ztp_temp.csv', index=False)
# Load the data from the CSV file
df = pd.read_csv('ztp_temp.csv')

# 2. Fit the Negative Binomial GLM
# Predictors: age, hmo (insurance type), died (binary outcome)
# Response: stay (number of days)
model = smf.glm('stay ~ age + hmo + died', data=df,
               family=sm.families.NegativeBinomial()).fit()

# 3. Model Summary
print(model.summary())

# 4. Calculate Incident Rate Ratios (IRR)
# IRR = exp(coefficient)
irr = np.exp(model.params)
print("\n--- Incident Rate Ratios (IRR) ---")
print(irr)
```

Generalized Linear Model Regression Results						
=====						
Dep. Variable:	stay	No. Observations:	1493			
Model:	GLM	Df Residuals:	1489			
Model Family:	NegativeBinomial	Df Model:	3			
Link Function:	Log	Scale:	1.0000			
Method:	IRLS	Log-Likelihood:	-4955.3			
Date:	Sat, 16 May 2026	Deviance:	827.73			
Time:	14:24:56	Pearson chi2:	938.			
No. Iterations:	5	Pseudo R-squ. (CS):	0.01154			
Covariance Type:	nonrobust					
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	2.4378	0.091	26.920	0.000	2.260	2.615
age	-0.0147	0.016	-0.894	0.371	-0.047	0.018
hmo	-0.1375	0.075	-1.845	0.065	-0.284	0.009
died	-0.2038	0.058	-3.505	0.000	-0.318	-0.090
=====						

--- Incident Rate Ratios (IRR) ---

Intercept 11.447342
age 0.985360
hmo 0.871547
died 0.815615

dtype: float64

C:\ProgramData\anaconda3\Lib\site-packages\statsmodels\genmod\families\family.py:136
7: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default
value alpha=1.0.
warnings.warn("Negative binomial dispersion parameter alpha not "

In []:

Problem 11: A study of length of hospital stay, in days, as a function of age, kind of health insurance (HMO), and whether or not the patient died while in the hospital. Length of hospital stay is recorded as a minimum of at least one day. The data set is taken from <https://stats.idre.ucla.edu/stat/data/ztp.dta>. Fit the zero-truncated negative binomial regression generalized linear models (GLMs) to identify the factors associated with length of hospital stay. Interpret the result.

```
In [1]: import pandas as pd
import numpy as np
import statsmodels.api as sm
import pyreadstat

from scipy.stats import nbinom
from statsmodels.base.model import GenericLikelihoodModel

# Load dataset
df, meta = pyreadstat.read_dta("ztp.dta")

# Variables
y = df["stay"]
X = sm.add_constant(df[["age", "hmo", "died"]])

# Custom ZTNB Model
class ZTNB(GenericLikelihoodModel):
    def loglike(self, params):
        beta = params[:-1]
        alpha = params[-1]

        if alpha <= 0:
            return -np.inf

        mu = np.exp(np.dot(self.exog, beta))

        size = 1 / alpha
        prob = size / (size + mu)

        ll_nb = nbinom.logpmf(self.endog, size, prob)

        p0 = (1 + alpha * mu) ** (-1 / alpha)

        return np.sum(ll_nb - np.log(1 - p0))

# Starting params
start_params = np.append(
    sm.GLM(y, X, family=sm.families.NegativeBinomial()).fit().params.values,
    1.0
)

# Fit model
```

```

model = ZTNB(y, X).fit(start_params=start_params)

# Result
print(model.summary())

# IRR
irr = np.exp(model.params[:-1])

print("\nIRR:")
print(pd.DataFrame({
    "Variable": X.columns,
    "IRR": irr
}))

# Interpretation (simple)
print("\nInterpretation:")

for var, irr_val in zip(X.columns, irr):
    if var == "const":
        continue

    if irr_val > 1:
        print(f"{var} increases stay by {(irr_val-1)*100:.2f}%")
    else:
        print(f"{var} decreases stay by {(1-irr_val)*100:.2f}%")

```

C:\ProgramData\anaconda3\Lib\site-packages\statsmodels\genmod\family.py:136
 7: ValueWarning: Negative binomial dispersion parameter alpha not set. Using default
 value alpha=1.0.
 warnings.warn("Negative binomial dispersion parameter alpha not ")

Optimization terminated successfully.

Current function value: 3.185050

Iterations: 248

Function evaluations: 408

ZTNB Results

```
=====
Dep. Variable:          stay    Log-Likelihood:          -4755.3
Model:                  ZTNB    AIC:                      9521.
Method:                Maximum Likelihood    BIC:              9547.
Date:                  Sat, 16 May 2026
Time:                  14:40:57
No. Observations:      1493
Df Residuals:          1489
Df Model:               3
=====
```

	coef	std err	z	P> z	[0.025	0.975]
const	2.4083	0.072	33.457	0.000	2.267	2.549
age	-0.0157	0.013	-1.198	0.231	-0.041	0.010
hmo	-0.1471	0.059	-2.484	0.013	-0.263	-0.031
died	-0.2177	0.046	-4.717	0.000	-0.308	-0.127
par0	0.5663	0.031	18.132	0.000	0.505	0.628

IRR:

	Variable	IRR
0	const	11.115182
1	age	0.984424
2	hmo	0.863217
3	died	0.804347

Interpretation:

age decreases stay by 1.56%

hmo decreases stay by 13.68%

died decreases stay by 19.57%

```
C:\ProgramData\anaconda3\Lib\site-packages\statsmodels\base\model.py:2748: UserWarni
ng: df_model + k_constant + k_extra differs from k_params
  warnings.warn("df_model + k_constant + k_extra ")
C:\ProgramData\anaconda3\Lib\site-packages\statsmodels\base\model.py:2752: UserWarni
ng: df_resid differs from nobs - k_params
  warnings.warn("df_resid differs from nobs - k_params")
```

In []: